

LAGOS STATE UNIVERSITY

FACULTY OF COMPUTING

DEPARTMENT OF COMPUTER SCIENCE

DESIGN AND DOCUMENTATION OF THE DATABASE FOR THE LFZDC FLEET MANAGEMENT SYSTEM

A DATABASE PROJECT REPORT

Submitted to the Department of Computer Science, Faculty of Computing, Lagos State University, in partial fulfilment of the requirements for the award of a degree in Computer Science.

BY

AYASI BOLUATIFE ANTHONY

MATRIC NUMBER: 2305-9-1-0-4-5

SUPERVISOR: _____

SUBMISSION DATE: _____

DECLARATION

I, Ayasi Boluatife Anthony, declare that this database report documents the design and implementation submitted for academic assessment. The schema descriptions, diagrams, queries and restoration procedures are based on the accompanying LFZDC Fleet Management System database package. Sources used to support the database concepts are acknowledged in the references section.

Student Signature: _____

Date: _____

CERTIFICATION

This is to certify that the database project titled “Design and Documentation of the Database for the LFZDC Fleet Management System” was carried out by Ayasi Boluatife Anthony, matric number 2305-9-1-0-4-5, in the Department of Computer Science, Faculty of Computing, Lagos State University.

Supervisor: _____

Signature: _____

Date: _____

ACKNOWLEDGEMENTS

I acknowledge the academic guidance and feedback that supported this project. I also appreciate the operational context of Lekki Free Zone Development Company, which helped define the records, relationships and controls represented in the database. I am grateful to my lecturers, colleagues and family for their support during the analysis, implementation, testing and documentation of the system.

ABSTRACT

The LFZDC Fleet Management System requires a database that can connect vehicle, driver, assignment, trip, fuel, maintenance, incident, document, notification and import records without losing operational history. This report presents the analysis, design, implementation and evaluation of that database. PostgreSQL is used as the relational database engine through Supabase, which also provides authentication, object storage, REST access and server-side functions. The implemented schema contains fourteen application tables, foreign-key relationships, uniqueness constraints, JSONB audit fields, timestamp triggers and Row Level Security policies. Three canonical application roles separate system administration, operational record management and read-only access. A profile-protection trigger prevents non-administrators from changing authorization fields, while storage policies restrict file modification and deletion. The submission contains ten ordered SQL scripts that recreate the schema, supporting functions, policies, storage bucket and fictional demonstration records. The design was evaluated against data integrity, referential consistency, authorization, restoration order and reporting requirements. The result is a portable and inspectable PostgreSQL database package that supports the implemented fleet application while remaining suitable for academic assessment and future extension.

Keywords: PostgreSQL, Supabase, relational database, fleet management, Row Level Security, database restoration, data integrity.

TABLE OF CONTENTS

CHAPTER ONE: INTRODUCTION....1
1.1 Background of the Study....1
1.2 Statement of the Problem....1
1.3 Aim and Objectives....1
1.4 Scope and Limitations....2
1.5 Methodology....2
CHAPTER TWO: REQUIREMENTS AND CONCEPTUAL DESIGN....3
2.1 Data Requirements....3
2.2 Business Rules....4
2.3 Entity Identification....4
2.4 Core Entity Relationships....5
2.5 Support and Governance Relationships....5
2.6 Normalization....6
CHAPTER THREE: LOGICAL AND PHYSICAL DESIGN....7
3.1 Relational Schema Overview....7
3.2 Core Table Design....7
3.3 Supporting Table Design....8
3.4 Keys, Constraints and Delete Behaviour....8
3.5 Data Types and JSONB Use....9
3.6 Indexing and Performance....9

CONTENTS (CONTINUED)

CHAPTER FOUR: DATABASE IMPLEMENTATION....10
4.1 PostgreSQL and Supabase Environment....10
4.2 Scripted Restoration Package....10
4.3 Functions and Triggers....11
4.4 Row Level Security....12
4.5 Storage Integration....12
4.6 Demonstration Data....13
CHAPTER FIVE: DATABASE OPERATIONS AND QUERIES....14
5.1 Application Data Access....14
5.2 Representative Queries....14
5.3 Transaction and Concurrency Considerations....15
5.4 Reporting and Import Audit....15
5.5 Practical Database Demonstration....16
5.6 Hosted Supabase SQL Evidence....23
CHAPTER SIX: TESTING AND EVALUATION....28
6.1 Verification Strategy....28
6.2 Integrity and Security Tests....28
6.3 Restoration Evaluation....29
6.4 Limitations....29
CHAPTER SEVEN: ADMINISTRATION AND RESTORATION GUIDE....30
7.1 Requirements and Safe Handling....30
7.2 Restoration Procedure....30
7.3 Verification and Maintenance....31
CHAPTER EIGHT: CONCLUSION AND RECOMMENDATIONS....32
8.1 Conclusion....32
8.2 Recommendations....32
REFERENCES....33
APPENDIX A: KEY DATA DICTIONARY....34
APPENDIX B: RESTORATION SCRIPT MANIFEST....36
APPENDIX C: SAMPLE SQL QUERIES....37

LIST OF FIGURES

Figure 2.1 Database platform context

Figure 2.2 Core fleet entity relationship model

Figure 2.3 Support and governance entity relationship model

Figure 4.1 Portable database restoration workflow

Figure 4.2 Database authorization flow

Figure 5.1 Creating a vehicle and returning the stored row

Figure 5.2 Reading and searching vehicle records

Figure 5.3 Updating a vehicle with before and after results

Figure 5.4 Joining vehicles, drivers and assignments

Figure 5.5 Calculating fuel and maintenance totals

Figure 5.6 Unique and foreign key constraint errors

Figure 5.7 Deleting a vehicle and verifying its removal

Figure 5.8 Hosted vehicle table structure in Supabase

Figure 5.9 Hosted vehicle status aggregation

Figure 5.10 Hosted current assignment join

Figure 5.11 Hosted assignment foreign key inspection

Figure 5.12 Hosted core-table Row Level Security flags

LIST OF TABLES

Table 1.1 Database project objectives

Table 2.1 Principal business rules

Table 2.2 Entity inventory

Table 2.3 Normalization decisions

Table 3.1 Core table summary

Table 3.2 Supporting table summary

Table 3.3 Relationship delete behaviour

Table 3.4 Data type rationale

Table 4.1 Restoration script responsibilities

Table 4.2 Database functions and triggers

Table 4.3 Role capability matrix

Table 4.4 Demonstration data inventory

Table 6.1 Database test cases

Table 7.1 Post-restoration checks

Table A.1 Key data dictionary

Table B.1 Script manifest

ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
DBMS	Database Management System
ERD	Entity Relationship Diagram
FK	Foreign Key
JSONB	Binary JSON data type in PostgreSQL
JWT	JSON Web Token
LFZDC	Lekki Free Zone Development Company
PK	Primary Key
RDBMS	Relational Database Management System
RLS	Row Level Security
SQL	Structured Query Language
UQ	Unique Constraint
UUID	Universally Unique Identifier

CHAPTER ONE

INTRODUCTION

This chapter introduces the database problem, project purpose, scope and development method.

1.1 Background of the Study

Fleet operations produce records that are meaningful only when their relationships are preserved. A fuel purchase belongs to a vehicle; a trip connects a vehicle and driver; a maintenance record changes the operational state of an asset; and an incident can affect cost, readiness and compliance. A database therefore provides more than storage. It establishes a controlled structure in which these facts can be connected, validated, searched and reported.

The LFZDC Fleet Management System uses PostgreSQL through Supabase. PostgreSQL supplies the relational engine, constraints, functions, triggers and Row Level Security. Supabase adds hosted authentication, Storage, REST access and Edge Functions while leaving the application data in a standard PostgreSQL database. This combination supports the web application and also permits the schema to be submitted as portable SQL files.

1.2 Statement of the Problem

Before centralization, related fleet facts may exist in separate spreadsheets, documents or informal records. Repeated vehicle details create inconsistent copies, deleted rows can erase useful history, and unrestricted editing makes it difficult to identify who should change operational data. A lecturer assessing the project also needs a database package that can be inspected and recreated without receiving live credentials or password hashes.

The database problem is therefore to represent the complete fleet domain with dependable relationships, preserve historical records, control access by role, support reporting and imports, and remain portable enough for academic restoration.

1.3 Aim and Objectives

The aim is to design and implement a secure relational database for the LFZDC Fleet Management System.

No.	Objective
1	Identify the entities and relationships required for fleet operations.
2	Define primary keys, foreign keys, uniqueness rules and suitable data types.

No.	Objective
3	Preserve assignments, odometer readings and other operational history.
4	Apply role-based database policies instead of relying only on interface controls.
5	Integrate Supabase Auth and Storage without exposing privileged credentials.
6	Provide realistic fictional data for safe demonstration and query testing.
7	Package the database as ordered SQL scripts that can be restored independently.

1.4 Scope and Limitations

The database covers profiles, vehicles, drivers, assignments, fuel logs, maintenance, incidents, trips, documents, document categories, notifications, email-delivery records, odometer history and import batches. It also documents the Storage bucket and the relationship between application profiles and Supabase Auth users.

The submitted package does not contain live user passwords, hosted binary files, private API keys or production credentials. It is not a physical copy of the hosted service; it is a schema-and-sample-data representation that recreates the implemented design. Geographic tracking, procurement approvals, payroll and external telematics are outside the current schema.

1.5 Methodology

Development followed an iterative database design process: requirements were extracted from application workflows; entities and business rules were identified; the conceptual relationships were converted into relational tables; constraints and policies were implemented; fictional data was added; and the restoration order was reviewed. The final audit also checked that no script referenced a function before that function was created and that later storage policies did not weaken role restrictions.

CHAPTER TWO

REQUIREMENTS AND CONCEPTUAL DESIGN

This chapter translates fleet operations into data requirements, entities, relationships and normalization decisions.

2.1 Data Requirements

The database must identify every vehicle and driver, retain operational and compliance facts, connect event records to their owners, and support filtered totals and histories. Authentication identity must remain separate from the application profile so that the hosted Auth service controls credentials while the database controls roles and business metadata.

- Vehicle data includes plate, type, make, model, year, cost, warranty, status and mileage.
- Driver data includes identity, contact, licence, employment dates and status.
- Operational events include assignments, trips, fuel, maintenance, incidents and odometer readings.
- Document records connect stored files to a vehicle or driver and may carry expiry dates.
- Governance records include user profiles, alerts, email-delivery deduplication and import audits.

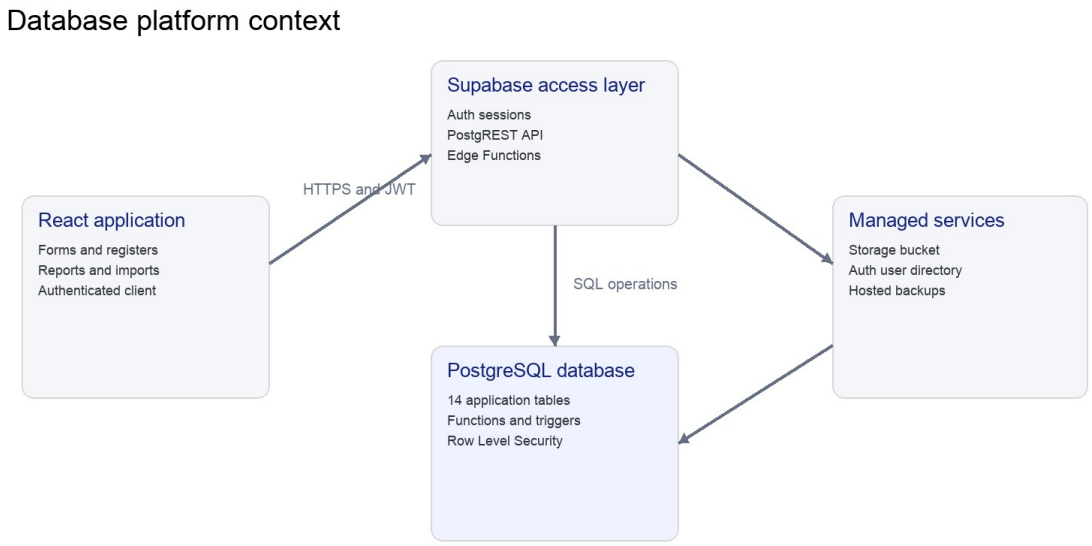


Figure 2.1: Database platform context. Source: Author's design, 2026.

2.2 Business Rules

Rule	Database interpretation
BR-01	A vehicle plate number is unique.
BR-02	A profile corresponds to one Supabase Auth user.
BR-03	An assignment must reference an existing vehicle and driver.
BR-04	A trip requires both a vehicle and a driver; an incident driver may be unknown.
BR-05	Deleting a vehicle removes dependent operational rows, while deleting an incident driver reference sets it to null.
BR-06	Only system administrators and editors may create or update operational records.
BR-07	Only system administrators may delete stored document objects.
BR-08	A non-administrator cannot change profile role or account status.
BR-09	Import outcomes retain aggregate counts and row-level issues in JSONB.
BR-10	The same notification-recipient pair can have only one recorded email delivery.

2.3 Entity Identification

Group	Entities	Purpose
Identity	profiles	Application role and user metadata linked to Auth.
Master data	vehicles, drivers	Stable records referenced by operational events.

Group	Entities	Purpose
Operations	assignments, trips, fuel_logs, maintenance, incidents	Day-to-day fleet activities and costs.
History	vehicle_odometer_logs, import_batches	Mileage provenance and migration audit.
Documents	documents, document_types	File metadata, categories and expiry tracking.
Communication	notifications, notification_email_deliveries	Alerts and email deduplication.

2.4 Core Entity Relationships

Vehicles and drivers are the central master entities. Assignment and trip records resolve their many-to-many operational relationship over time. Fuel, maintenance and odometer records belong to vehicles, while incidents may also identify a driver. Dependent rows use foreign keys so the database does not accept an event for an unknown asset.

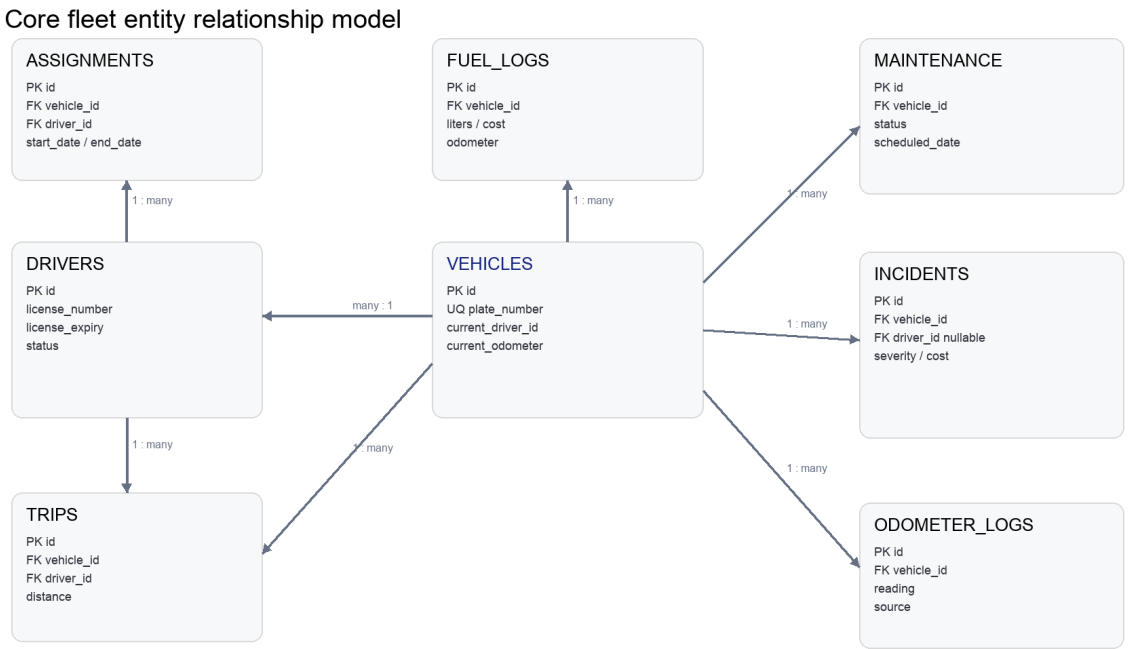


Figure 2.2: Core fleet entity relationship model. Source: Author's design, 2026.

2.5 Support and Governance Relationships

The profile table extends Supabase Auth without storing passwords. Notifications may reference a vehicle or driver. Email deliveries use a composite primary key to prevent duplicate delivery records. Documents store relational metadata while binary objects remain in Storage. Import batches retain the operator identity when available but use a null reference if that profile is later removed.

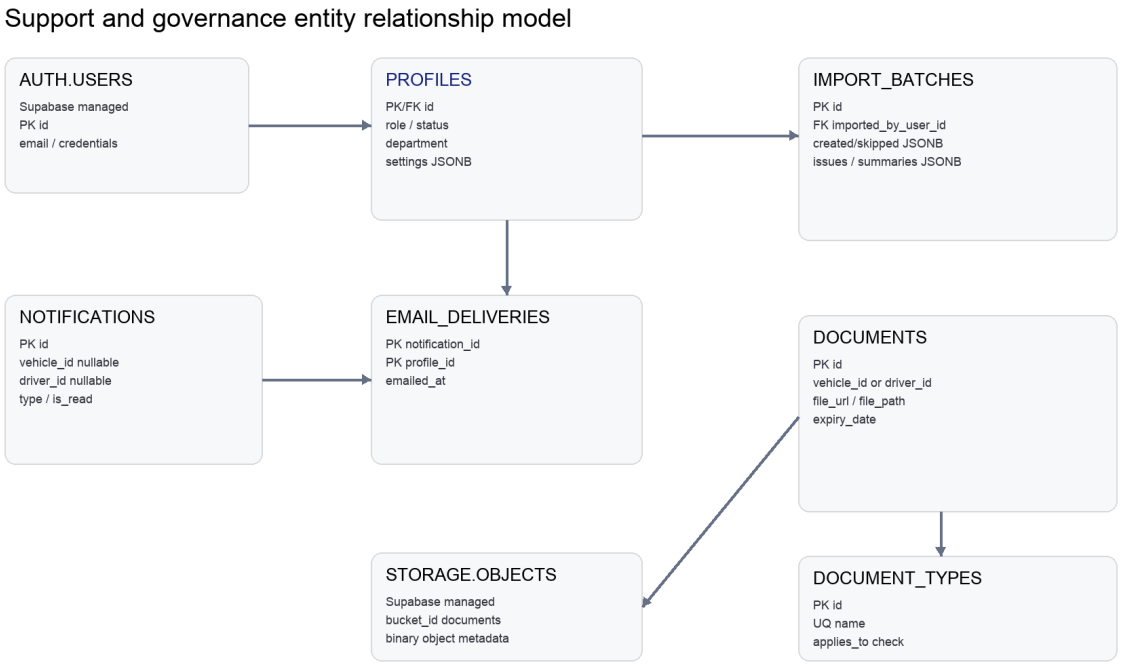


Figure 2.3: Support and governance entity relationship model. Source: Author's design, 2026.

2.6 Normalization

The operational schema largely follows third normal form. Repeating groups are separated into child tables, relationships use keys rather than duplicated descriptive fields, and non-key attributes describe the row identified by the primary key. Selected denormalization is intentional where it preserves an audit snapshot or improves operational access.

Design choice	Reason
Separate vehicles and drivers	Avoids repeating master details in every assignment, trip or incident.
Assignment history table	Represents relationship changes over time instead of overwriting one current-driver field.
Document type lookup	Standardizes categories while permitting controlled extension.
imported_by_name snapshot	Preserves the visible operator name even if a profile is later removed.
JSONB import details	Stores variable sheet summaries and row issues without creating sparse fixed columns.
vehicles.current_driver_id	Provides a fast operational pointer while assignments remain the historical source.

CHAPTER THREE

LOGICAL AND PHYSICAL DESIGN

This chapter documents the relational schema, table responsibilities, keys, constraints, data types and performance considerations.

3.1 Relational Schema Overview

The public schema contains fourteen application tables. Supabase-managed auth.users and storage.objects remain external dependencies rather than copied application tables. Text identifiers are used for most fleet records because the application generates readable stable IDs during normal forms and imports; UUIDs are retained for hosted user identities and document categories.

Module	Tables	Primary relationship
Identity	profiles	profiles.id → auth.users.id
Fleet	vehicles, drivers, assignments	assignments → vehicles and drivers
Operations	fuel_logs, maintenance, incidents, trips	each event → vehicle; selected events → driver
Documents	documents, document_types	documents → vehicle or driver; file_path → Storage
Audit	vehicle_odometer_logs, import_batches	odometer log → vehicle; import batch → profile
Alerts	notifications, notification_email_deliveries	delivery → notification and profile

3.2 Core Table Design

3.2.1 Vehicles and Drivers

These tables hold the stable master records from which operational relationships are formed.

Table	Primary key	Identifier or reference	Responsibility
vehicles	id	plate_number	Asset identity, commercial data, readiness and service mileage.
drivers	id	license_number	Driver identity, licence, employment and status data.

3.2.2 Assignments and Trips

Assignments preserve periods of responsibility, while trips record completed journeys and distances.

Table	Primary key	Identifier or reference	Responsibility
assignments	id	vehicle_id, driver_id	Start and optional end date for vehicle-driver allocation.
trips	id	vehicle_id, driver_id	Journey date, route, distance and purpose.

3.2.3 Fuel, Maintenance and Incidents

These event tables support cost reporting, readiness decisions and traceable operational history.

Table	Primary key	Identifier or reference	Responsibility
fuel_logs	id	vehicle_id	Litres, cost, station and odometer at purchase.
maintenance	id	vehicle_id	Service type, workflow state, dates, cost and provider.
incidents	id	vehicle_id, driver_id	Severity, workflow state, location and repair costs.

3.3 Supporting Table Design

Table	Key design	Responsibility
profiles	UUID PK and FK to auth.users	Application name, role, status, department and settings.
documents	Text PK; nullable vehicle/driver FKs	File metadata and expiry tracking.
document_types	UUID PK; unique name; applies_to check	Controlled shared document categories.
notifications	Text PK; nullable vehicle/driver FKs	Operational alert feed.
notification_email_deliveries	Composite PK	One delivery record per notification and recipient.
vehicle_odometer_logs	Text PK; vehicle FK	Reading provenance and source notes.
import_batches	Text PK; nullable profile FK	Workbook counts, issues and sheet summaries.

3.4 Keys, Constraints and Delete Behaviour

Primary keys provide row identity, foreign keys prevent orphaned events, and unique constraints protect natural identifiers such as profile email, vehicle plate and document type name. The database uses delete actions deliberately according to the lifecycle of each relationship.

Parent	Dependent relationship	Delete action	Reason
auth.users	profiles.id	CASCADE	Remove the application identity when the Auth account is removed.
vehicles	assignments, fuel, maintenance, trips, documents, notifications, odometer logs	CASCADE	Dependent fleet records have no meaning without the asset.
drivers	assignments and trips	CASCADE	Operational history is tightly linked to the driver record.
drivers	incidents.driver_id	SET NULL	Retain the incident even if the driver reference is unavailable.
profiles	import_batches.imported_by_user_id	SET NULL	Retain migration history after account removal.
notifications/profiles	notification_email_deliveries	CASCADE	Delivery audit belongs to both referenced records.

3.5 Data Types and JSONB Use

Type	Use	Rationale
text	Record IDs, labels and identifiers	Matches application-generated IDs and variable-length operational text.
uuid	Auth-linked profiles and document types	Compatible with Supabase identities and generated lookup keys.
date	Operational and expiry dates	Represents calendar dates without unintended time-zone conversion.
timestamptz	Creation, update and audit times	Stores absolute instants consistently in UTC.
numeric(p,s)	Fuel, distance and monetary values	Avoids binary floating-point error for measured and financial values.
integer/bigint	Mileage and file size	Efficient whole-number storage with appropriate range.
boolean	Notification read state	Represents a two-state fact directly.
jsonb	Import outcomes, attachments and preferences	Supports variable structured data and future indexing when needed.
app_role enum	Profile role	Restricts role values to known database-level labels.

3.6 Indexing and Performance

Primary and unique keys create indexes automatically, but foreign keys do not. Candidate indexes include vehicle-and-date lookups, assignment references and document expiry dates. Add indexes only when measured query plans justify their cost. Appendix C.5 lists two optional examples; these are not part of the restoration package.

CHAPTER FOUR

DATABASE IMPLEMENTATION

This chapter explains the PostgreSQL environment, restoration scripts, functions, triggers, security policies, Storage integration and sample data.

4.1 PostgreSQL and Supabase Environment

The implementation runs on PostgreSQL provided by Supabase. The frontend accesses tables through PostgREST using the authenticated user's JWT. The anonymous browser key identifies the project but does not bypass RLS. Privileged Edge Functions use the service-role credential only in the hosted server environment. Database passwords, service keys and SMTP credentials are excluded from the submission.

4.2 Scripted Restoration Package

The database-submission folder contains ten numbered scripts. Their order is significant because later scripts depend on objects created earlier. The package audit moved document-type write authorization into script 08, after the role helper functions exist, and ensures script 09 leaves Storage with the intended role restrictions.

Order	Script	Responsibility
01	01-schema.sql	Core enum, tables, timestamp function, triggers, RLS and read policies.
02	02-vehicle-odometer.sql	Current odometer field, odometer history table and read policy.
03	03-vehicle-service-mileage.sql	Next-service mileage field.
04	04-document-types.sql	Document category table and authenticated read policy.
05	05-document-file-path.sql	Storage object path and transitional document policy.
06	06-profile-settings.sql	JSONB settings on profiles.
07	07-notification-email-deliveries.sql	Email delivery deduplication table.
08	08-role-access-policies.sql	Canonical roles, helper functions, safeguards and final table write policies.
09	09-storage-documents.sql	Documents bucket and final object access policies.
10	10-sample-data.sql	Fictional data for assessment and demonstrations.

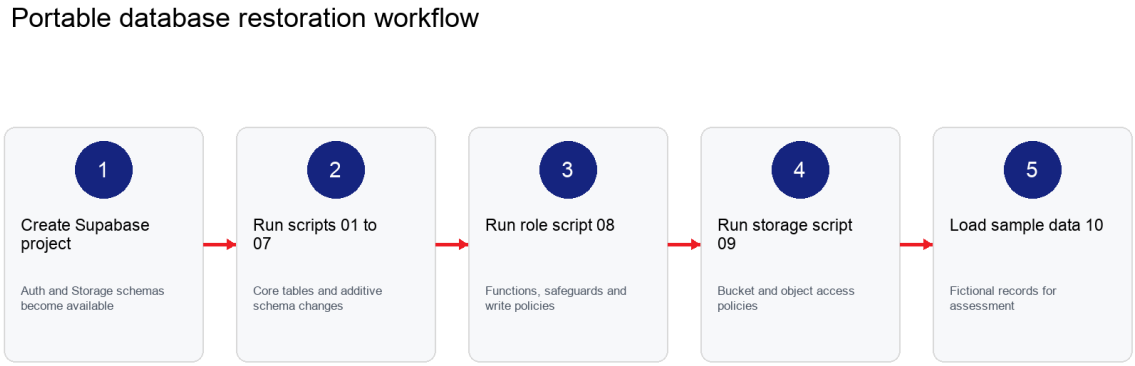


Figure 4.1: Portable database restoration workflow. Source: Author's design, 2026.

4.3 Functions and Triggers

Object	Type	Purpose
set_updated_at	Trigger function	Writes the current UTC timestamp before updates.
current_app_role	Security-definer SQL function	Returns the signed-in profile role without recursive RLS.
is_system_admin	Security-definer SQL function	Checks administrator capability.
can_manage_records	Security-definer SQL function	Allows system_admin and editor write operations.
protect_profile_authorization_fields	Trigger function	Blocks non-admin changes to role and account status.

The update timestamp trigger is attached to profiles, vehicles, drivers, maintenance, incidents and documents. The profile authorization trigger executes before every profile update. Service-role operations remain possible because server-side administration does not carry an ordinary end-user auth.uid value.

```
if auth.uid() is not null
   and not public.is_system_admin()
   and (new.role is distinct from old.role
        or new.status is distinct from old.status) then
  raise exception 'Only system administrators can change profile roles or
account status';
end if;
```

4.4 Row Level Security

RLS is enabled on every application table. Read policies allow authenticated staff to view operational information, while write policies call role helpers. The interface mirrors these permissions for usability, but the database remains the enforcement boundary.



Figure 4.2: Database authorization flow. Source: Author's design, 2026.

Capability	System admin	Editor	End user
Read fleet records	Yes	Yes	Yes
Create/update fleet records	Yes	Yes	No
Create document types	Yes	Yes	No
Insert import batches	Yes	Yes	No
View all profiles	Yes	Own profile	Own profile
Change another profile role/status	Yes	No	No
Upload/update stored files	Yes	Yes	No
Delete stored files	Yes	No	No

4.5 Storage Integration

Document metadata is stored in public.documents while binary files are stored in the Supabase documents bucket. Authenticated users may read objects, editors and administrators may upload or update them, and only administrators may delete them. The current bucket is public because the application uses public URLs. A production deployment handling confidential documents should convert the bucket to private access and use short-lived signed URLs.

4.6 Demonstration Data

Script 10 inserts fictional operational records only. It does not create Auth users, password hashes or real files. Conflict handling makes the inserts repeatable for stable IDs and unique document type names.

Dataset	Demonstration coverage
Vehicles and drivers	Multiple asset classes, statuses, mileage levels and licence dates.
Assignments	Current and historical vehicle-driver allocations.
Fuel and trips	March 2026 transactions for expenditure and distance reporting.
Maintenance and incidents	Completed, scheduled, in-progress, overdue and under-review cases.
Documents	Vehicle and driver categories with optional expiry dates.
Notifications	Maintenance, document, incident, fuel and licence alerts.
Odometer history	Manual, trip, fuel and maintenance reading sources.

CHAPTER FIVE

DATABASE OPERATIONS AND QUERIES

This chapter presents the main access patterns, representative SQL, transaction considerations and reporting support.

5.1 Application Data Access

The application does not maintain a separate custom REST server for ordinary records. The Supabase JavaScript client sends authenticated requests to PostgREST, which converts allowed operations into SQL against the public schema. Edge Functions are reserved for privileged account administration and email delivery. This separation keeps routine data access direct while protecting service-role operations.

5.2 Representative Queries

5.2.1 Fleet Readiness Summary

```
select status, count(*) as vehicle_count
from public.vehicles
group by status
order by status;
```

5.2.2 Vehicle Operating Cost

```
select v.plate_number,
       coalesce(f.total, 0) as fuel_cost,
       coalesce(m.total, 0) as maintenance_cost
from public.vehicles v
left join (
  select vehicle_id, sum(cost) as total
  from public.fuel_logs group by vehicle_id
) f on f.vehicle_id = v.id
left join (
  select vehicle_id, sum(cost) as total
  from public.maintenance group by vehicle_id
) m on m.vehicle_id = v.id
order by v.plate_number;
```

Each event table is aggregated to one row per vehicle before the joins. This prevents fuel and maintenance rows from multiplying each other and inflating the totals. COALESCE returns zero for a vehicle with no matching costs. Section 5.5 demonstrates the query using two fuel entries and two maintenance entries for the same vehicle.

5.2.3 Current Assignments

```
select v.plate_number, d.full_name, a.start_date
from public.assignments a
join public.vehicles v on v.id = a.vehicle_id
join public.drivers d on d.id = a.driver_id
where a.end_date is null
order by a.start_date desc;
```

5.2.4 Expiring Documents

```
select title, document_type, expiry_date
from public.documents
where expiry_date between current_date and current_date + interval '60 days'
order by expiry_date;
```

5.2.5 Import Audit Review

```
select file_name, imported_at, imported_by_name,
       total_rows, valid_rows, issue_count
from public.import_batches
order by imported_at desc;
```

5.3 Transaction and Concurrency Considerations

Several workflows change related rows. Driver assignment closes conflicting active assignments, inserts a new assignment and updates the vehicle pointer. Maintenance and odometer workflows can also update both an event row and the vehicle summary state. The current application sequences these calls; future hardening should move multi-step invariants into PostgreSQL functions executed as one transaction so concurrent operators cannot create an inconsistent intermediate state.

Uniqueness constraints already protect profile email, vehicle plate and document type name. Assignment exclusivity is currently enforced by application logic. A partial unique index on active assignments would provide stronger database enforcement if the operational rule remains one active vehicle per driver and one active driver per vehicle.

```
create unique index on public.assignments (vehicle_id)
where end_date is null;

create unique index on public.assignments (driver_id)
where end_date is null;
```

5.4 Reporting and Import Audit

The report module filters operational events by vehicle and date range, then calculates fuel cost, maintenance cost, incident cost and distance. Import batches retain aggregate counts and row-level issues so a historical workbook can be reviewed after migration. JSONB is appropriate because sheet summaries and errors have variable structures, but important filter fields such as file name, operator and timestamp remain normal columns.

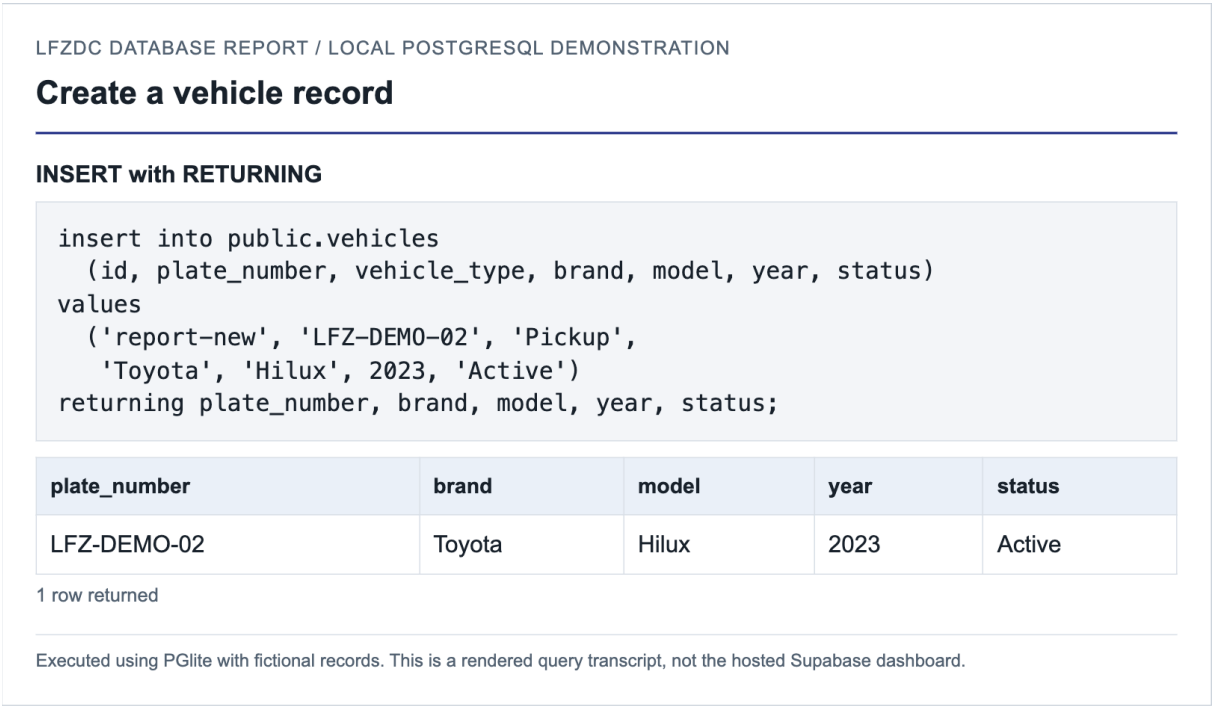
5.5 Practical Database Demonstration

The following walkthrough shows the input SQL and actual database response for Create, Read, Update, Delete and search operations, followed by relationship and integrity checks. The sequence uses fictional LFZ-DEMO records and does not modify the hosted fleet database.

Evidence was executed in isolated PostgreSQL 18.3 through PGlite 0.5.8. Seven core table definitions and the vehicle timestamp trigger were taken from 01-schema.sql. The figures are screenshots of rendered execution transcripts, not screenshots of the Supabase dashboard. They verify the demonstrated SQL behaviour, not hosted authentication, RLS policies, Storage or a complete restoration.

5.5.1 Create a Vehicle

Enter a unique record ID and plate number, then supply the vehicle type, brand, model, year and status. INSERT stores the row; RETURNING displays the stored values immediately. In the application, the Add Vehicle form reaches the equivalent insert through createVehicle in src/lib/fleet-data.ts.


LFZDC DATABASE REPORT / LOCAL POSTGRESQL DEMONSTRATION

Create a vehicle record

INSERT with RETURNING

```
insert into public.vehicles
  (id, plate_number, vehicle_type, brand, model, year, status)
values
  ('report-new', 'LFZ-DEMO-02', 'Pickup',
   'Toyota', 'Hilux', 2023, 'Active')
returning plate_number, brand, model, year, status;
```

plate_number	brand	model	year	status
LFZ-DEMO-02	Toyota	Hilux	2023	Active

1 row returned

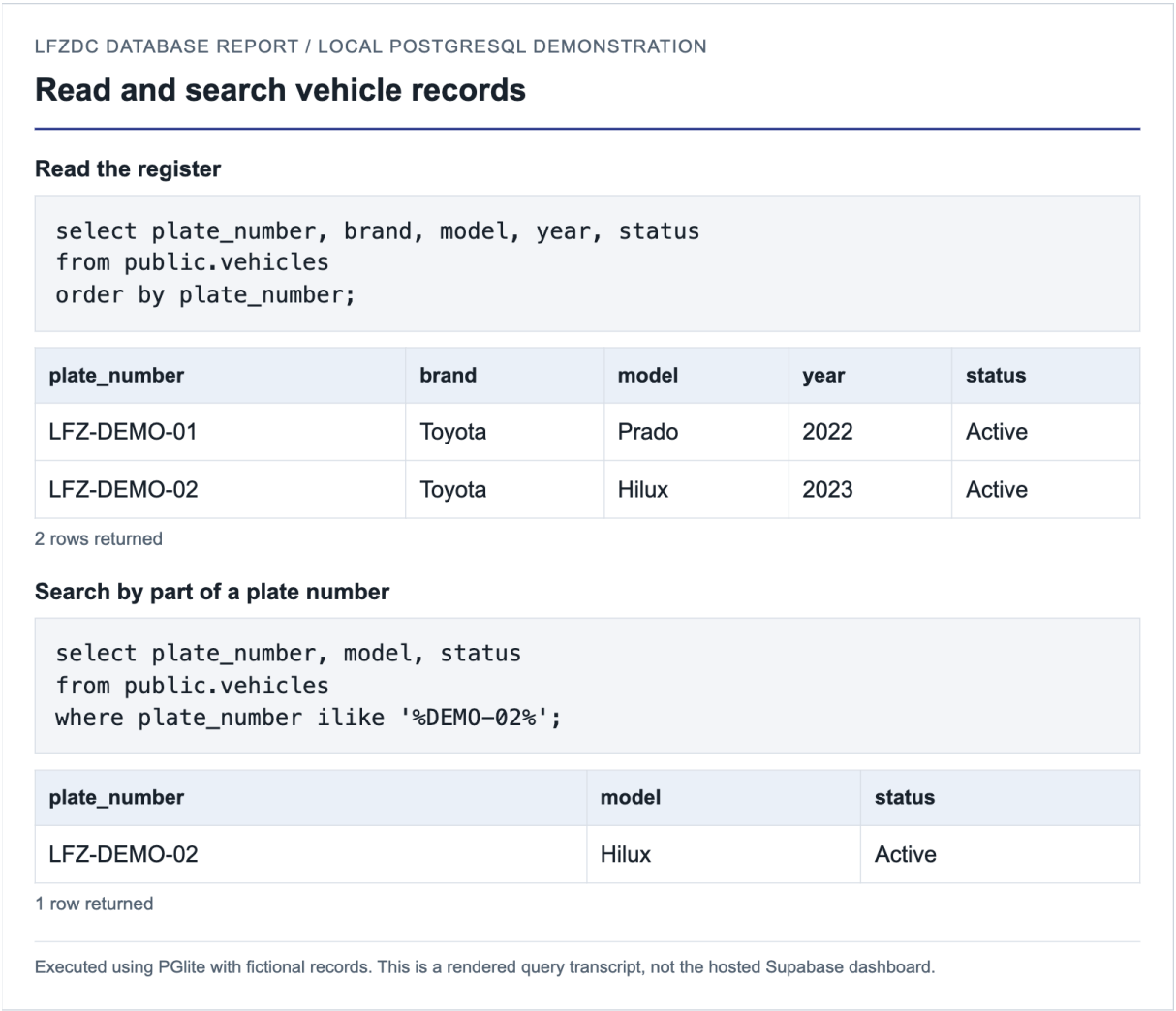
Executed using PGlite with fictional records. This is a rendered query transcript, not the hosted Supabase dashboard.

Figure 5.1: Creating a vehicle and returning the stored row

The returned LFZ-DEMO-02 row confirms that the Toyota Hilux was inserted with year 2023 and status Active. Its ID, report-new, is used in the later update and delete operations so that only this demonstration record is changed.

5.5.2 Read and Search Records

Run SELECT to display the vehicle register, then add a WHERE condition to narrow the result. The first query returns the original demonstration vehicle and the newly inserted vehicle. The second uses ILIKE for a case-insensitive partial plate-number search.



LFZDC DATABASE REPORT / LOCAL POSTGRESQL DEMONSTRATION

Read and search vehicle records

Read the register

```
select plate_number, brand, model, year, status
from public.vehicles
order by plate_number;
```

plate_number	brand	model	year	status
LFZ-DEMO-01	Toyota	Prado	2022	Active
LFZ-DEMO-02	Toyota	Hilux	2023	Active

2 rows returned

Search by part of a plate number

```
select plate_number, model, status
from public.vehicles
where plate_number ilike '%DEMO-02%';
```

plate_number	model	status
LFZ-DEMO-02	Hilux	Active

1 row returned

Executed using PGlite with fictional records. This is a rendered query transcript, not the hosted Supabase dashboard.

Figure 5.2: Reading and searching vehicle records

The full read returns two rows. Searching for DEMO-02 returns only the Hilux. These operations retrieve records without changing them; the percent signs allow text before or after the search term.

5.5.3 Update a Vehicle

Read the selected record before editing it. Change its status from Active to Under Maintenance and retain the WHERE condition on its primary key. RETURNING makes the new value visible without requiring a separate refresh.

LFZDC DATABASE REPORT / LOCAL POSTGRESQL DEMONSTRATION

Update a specific vehicle

Before the change

```
select plate_number, status
from public.vehicles where id = 'report-new';
```

plate_number	status
LFZ-DEMO-02	Active

1 row returned

Update only the selected record

```
update public.vehicles
set status = 'Under Maintenance'
where id = 'report-new'
returning plate_number, status;
```

plate_number	status
LFZ-DEMO-02	Under Maintenance

1 row returned

Executed using PGlite with fictional records. This is a rendered query transcript, not the hosted Supabase dashboard.

Figure 5.3: Updating a vehicle with before and after results

The same plate number appears before and after the change, while the status changes. A separate automated assertion also confirmed that the vehicle updated_at trigger advanced the timestamp. Without the WHERE condition, an UPDATE would affect every row in the table.

5.5.4 Retrieve Related Records

The assignment fixture links vehicle report-base to driver report-driver from 1 September 2026. Rather than storing another copy of the plate number and driver name in the assignment, the query joins their IDs to the corresponding parent records.

LFZDC DATABASE REPORT / LOCAL POSTGRESQL DEMONSTRATION

Retrieve related fleet records

Join assignments to vehicles and drivers

```
select v.plate_number, d.full_name,
       a.start_date::text as start_date
from public.assignments a
join public.vehicles v on v.id = a.vehicle_id
join public.drivers d on d.id = a.driver_id
where a.end_date is null;
```

plate_number	full_name	start_date
LFZ-DEMO-01	Demonstration Driver	2026-09-01

1 row returned

Executed using PGlite with fictional records. This is a rendered query transcript, not the hosted Supabase dashboard.

Figure 5.4: Joining vehicles, drivers and assignments

The result associates LFZ-DEMO-01 with Demonstration Driver. The condition end_date IS NULL restricts the result to an assignment that has not been closed. An inner join requires the referenced vehicle and driver to exist.

This example illustrates why the database separates vehicles, drivers and assignments: the assignment stores the relationship and its dates, while the vehicle and driver tables remain the authoritative sources for their own details.

5.5.5 Summarize Operating Costs

The fixture contains fuel costs of NGN 20,000 and NGN 30,000, plus maintenance costs of NGN 10,000 and NGN 15,000 for LFZ-DEMO-01. Aggregate the tables separately before combining them by vehicle ID.

LFZDC DATABASE REPORT / LOCAL POSTGRESQL DEMONSTRATION

Calculate costs without double counting

Aggregate each event table before joining

```
select v.plate_number,
       coalesce(f.total, 0) as fuel_cost,
       coalesce(m.total, 0) as maintenance_cost
from public.vehicles v
left join (
  select vehicle_id, sum(cost) as total
  from public.fuel_logs group by vehicle_id
) f on f.vehicle_id = v.id
left join (
  select vehicle_id, sum(cost) as total
  from public.maintenance group by vehicle_id
) m on m.vehicle_id = v.id
order by v.plate_number;
```

plate_number	fuel_cost	maintenance_cost
LFZ-DEMO-01	50000.00	25000.00
LFZ-DEMO-02	0	0

2 rows returned

Executed using PGlite with fictional records. This is a rendered query transcript, not the hosted Supabase dashboard.

Figure 5.5: Calculating fuel and maintenance totals

The correct totals are NGN 50,000 for fuel and NGN 25,000 for maintenance. The second vehicle has no cost records and returns zero. Joining the two raw event tables first would produce four combinations and incorrectly double both totals for LFZ-DEMO-01.

5.5.6 Demonstrate Integrity Constraints

The following invalid inputs were deliberately submitted to the isolated database. The first repeats an existing plate number. The second creates a trip with a vehicle ID that is absent from the vehicle table. Both operations must fail.

LFZDC DATABASE REPORT / LOCAL POSTGRESQL DEMONSTRATION

Reject invalid records

Duplicate plate number

```
insert into public.vehicles
  (id, plate_number, vehicle_type, brand, model, year, status)
values ('report-duplicate', 'LFZ-DEMO-01', 'SUV',
        'Toyota', 'Prado', 2022, 'Active');
```

SQLSTATE 23505

duplicate key value violates unique constraint "vehicles_plate_number_key"

Key (plate_number)=(LFZ-DEMO-01) already exists.

Trip referencing an unknown vehicle

```
insert into public.trips
  (id, vehicle_id, driver_id, date,
   start_location, destination, distance)
values ('report-bad-trip', 'missing-vehicle',
        'report-driver', '2026-09-17',
        'Gate A', 'Warehouse', 8);
```

SQLSTATE 23503

insert or update on table "trips" violates foreign key constraint "trips_vehicle_id_fkey"

Key (vehicle_id)=(missing-vehicle) is not present in table "vehicles".

Executed using PGlite with fictional records. This is a rendered query transcript, not the hosted Supabase dashboard.

Figure 5.6: Unique and foreign key constraint errors

SQLSTATE 23505 identifies the duplicate plate-number violation. SQLSTATE 23503 identifies the invalid vehicle reference. These are expected test results, not unresolved application faults; neither invalid row was stored.

5.5.7 Delete and Verify Removal

Delete only the temporary report-new vehicle. The RETURNING clause displays the removed record, and a subsequent count verifies that no row with that ID remains. This final step completes the Create, Read, Update and Delete sequence.



LFZDC DATABASE REPORT / LOCAL POSTGRESQL DEMONSTRATION

Delete and verify the demonstration record

Delete only the newly created vehicle

```
delete from public.vehicles
where id = 'report-new'
returning plate_number, status;
```

plate_number	status
LFZ-DEMO-02	Under Maintenance

1 row returned

Check that the selected record is gone

```
select count(*) as remaining_rows
from public.vehicles where id = 'report-new';
```

remaining_rows
0

1 row returned

Executed using PGlite with fictional records. This is a rendered query transcript, not the hosted Supabase dashboard.

Figure 5.7: Deleting a vehicle and verifying its removal

The returned row is LFZ-DEMO-02 with its updated Under Maintenance status. The verification count is zero. The original report-base vehicle is not targeted by this statement.

In a real project, review related records and take an appropriate backup before deleting a vehicle. Several child tables use ON DELETE CASCADE, so deletion can remove its operational history. These demonstrations use only disposable fictional records in a separate database.

5.6 Hosted Supabase SQL Evidence

Figures 5.8 to 5.12 are actual screenshots of the hosted project SQL Editor, captured on 17 September 2026. Each shows the executed SELECT statement and its returned results. These read-only checks did not create, update or delete live records. Counts reflect the database at capture time, not the fictional local fixtures in Section 5.5.

5.6.1 Inspect the Vehicle Table

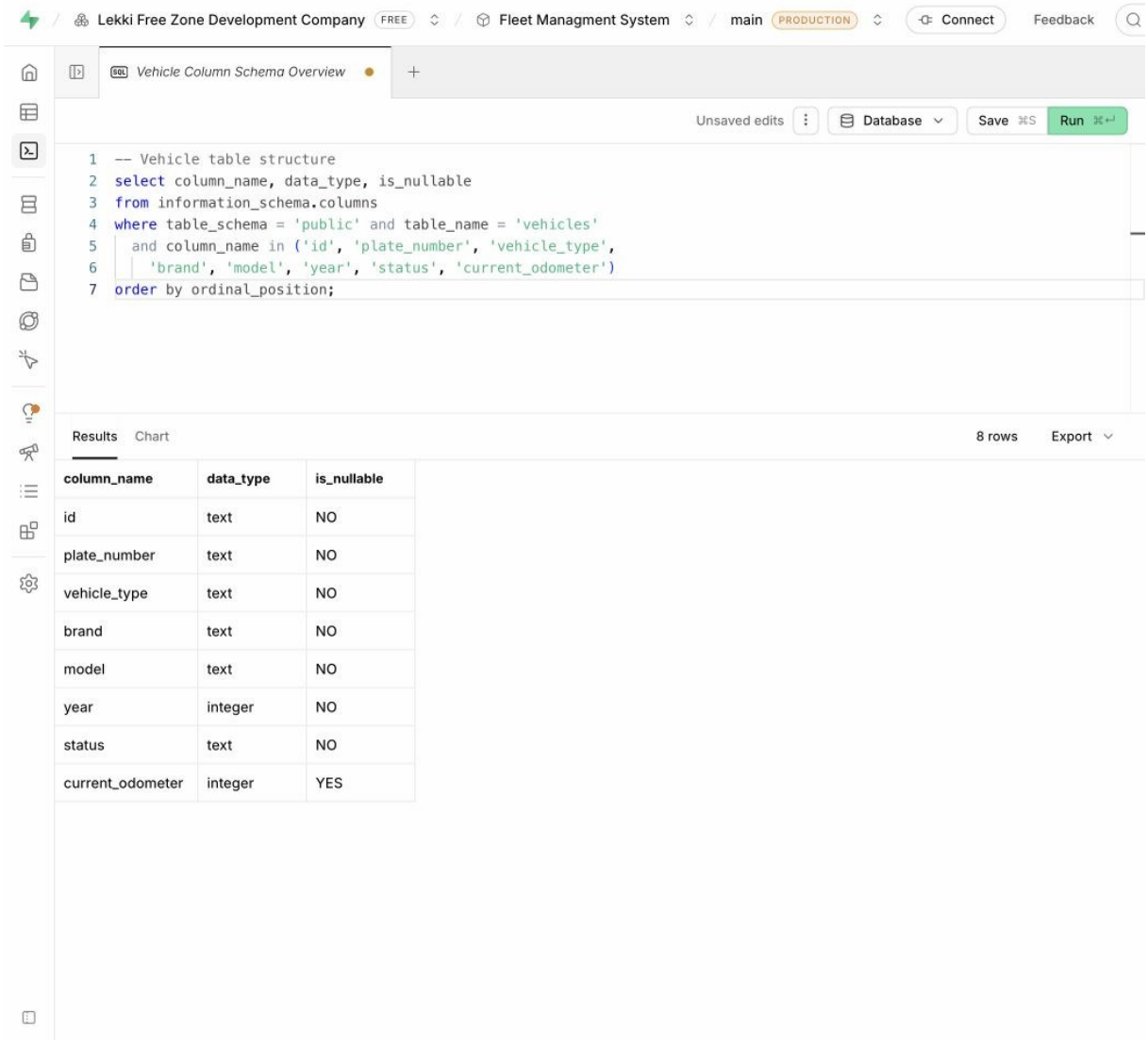


Figure 5.8: Hosted vehicle table structure in Supabase

The information_schema query returns eight selected columns with their data types and nullability. Seven are mandatory; current_odometer permits NULL. This confirms the inspected fields in the deployed table rather than merely reproducing a proposed schema.

5.6.2 Summarize the Live Vehicle Register

COUNT and GROUP BY reduce the vehicle register to one row per status. The statement is useful for a dashboard summary because it returns counts instead of the complete register. ORDER BY makes the output predictable.

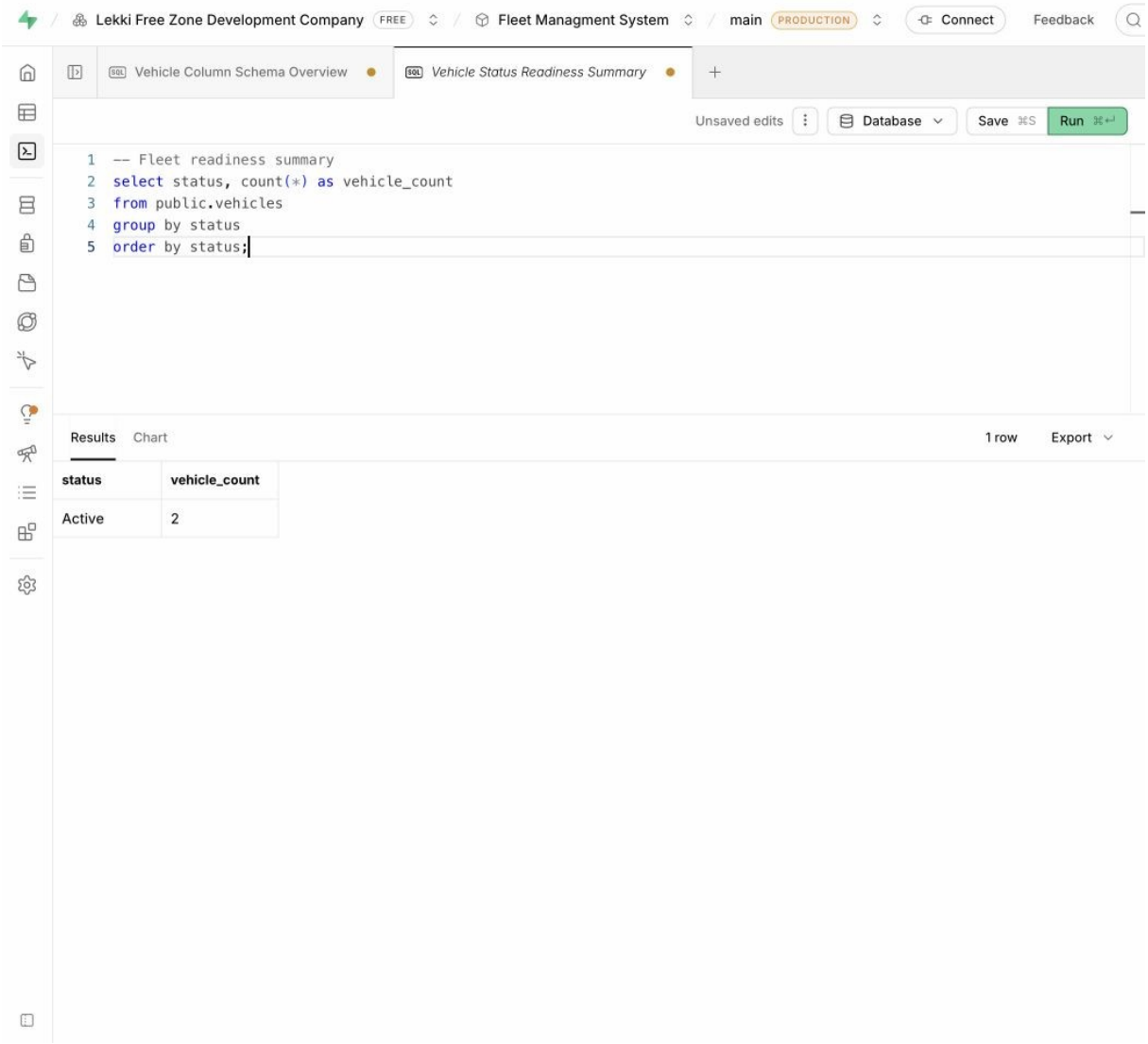


Figure 5.9: Hosted vehicle status aggregation

The live query returns Active with a vehicle_count of 2. Only statuses represented by stored rows appear; the absence of another status does not mean that status is invalid. The result is a point-in-time observation and can change when records are edited.

5.6.3 Join Vehicles, Drivers and Assignments

This statement follows the assignment foreign keys to the vehicle and driver tables. It displays the vehicle plate, model, driver status and assignment start date. Driver names, contact details and licence numbers are intentionally excluded from this report evidence.

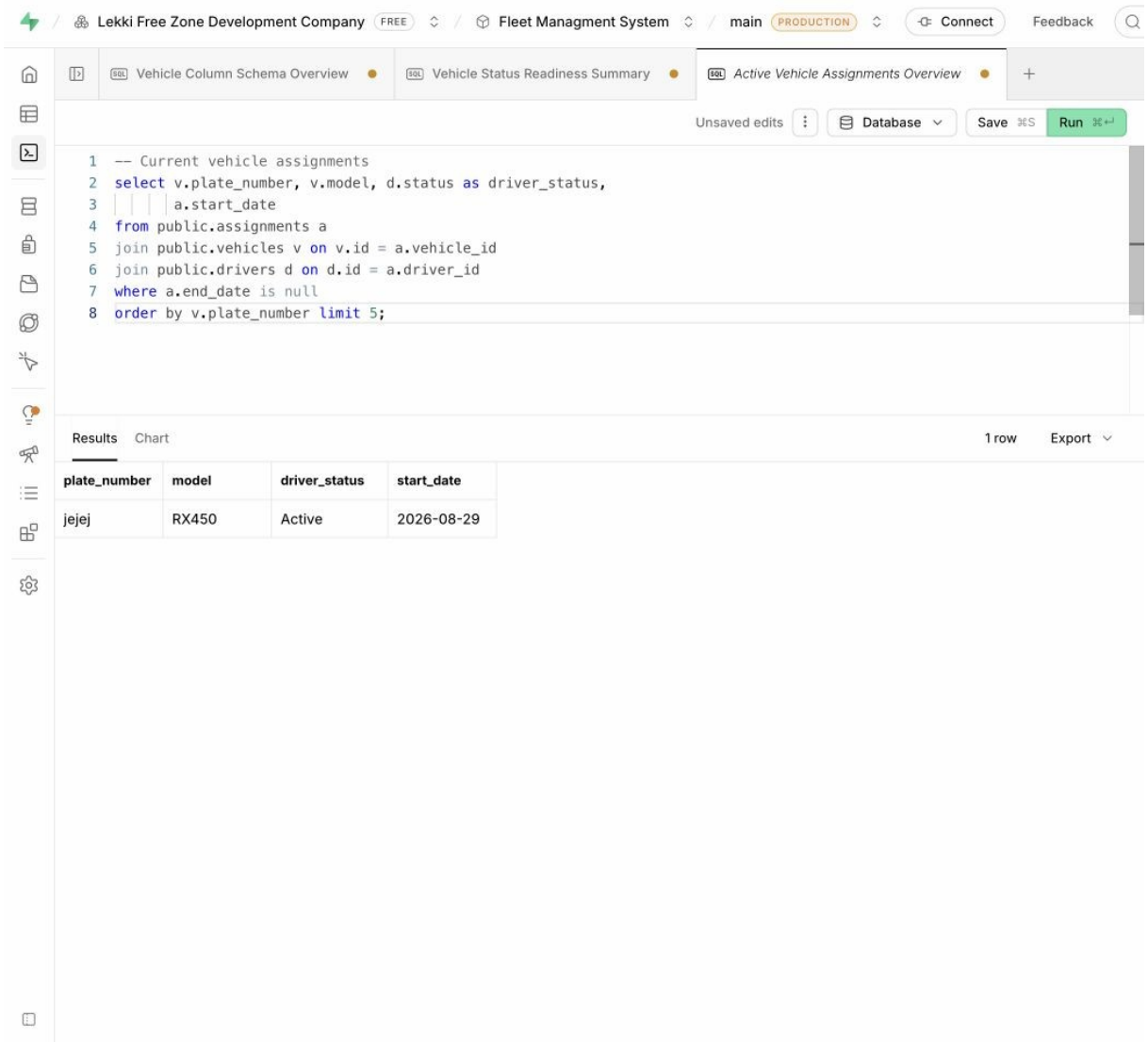


Figure 5.10: Hosted current assignment join

One current assignment is returned: vehicle plate jejej, model RX450, driver status Active and start date 2026-08-29. The condition end_date IS NULL selects open assignments. LIMIT 5 caps the displayed rows; the result contains one matching row at capture time.

5.6.4 Inspect Referential Integrity Rules

The PostgreSQL pg_constraint catalogue records the constraints installed in the database. This query selects foreign keys on public.assignments, identifies their referenced tables and translates the cascade delete code into a readable label.

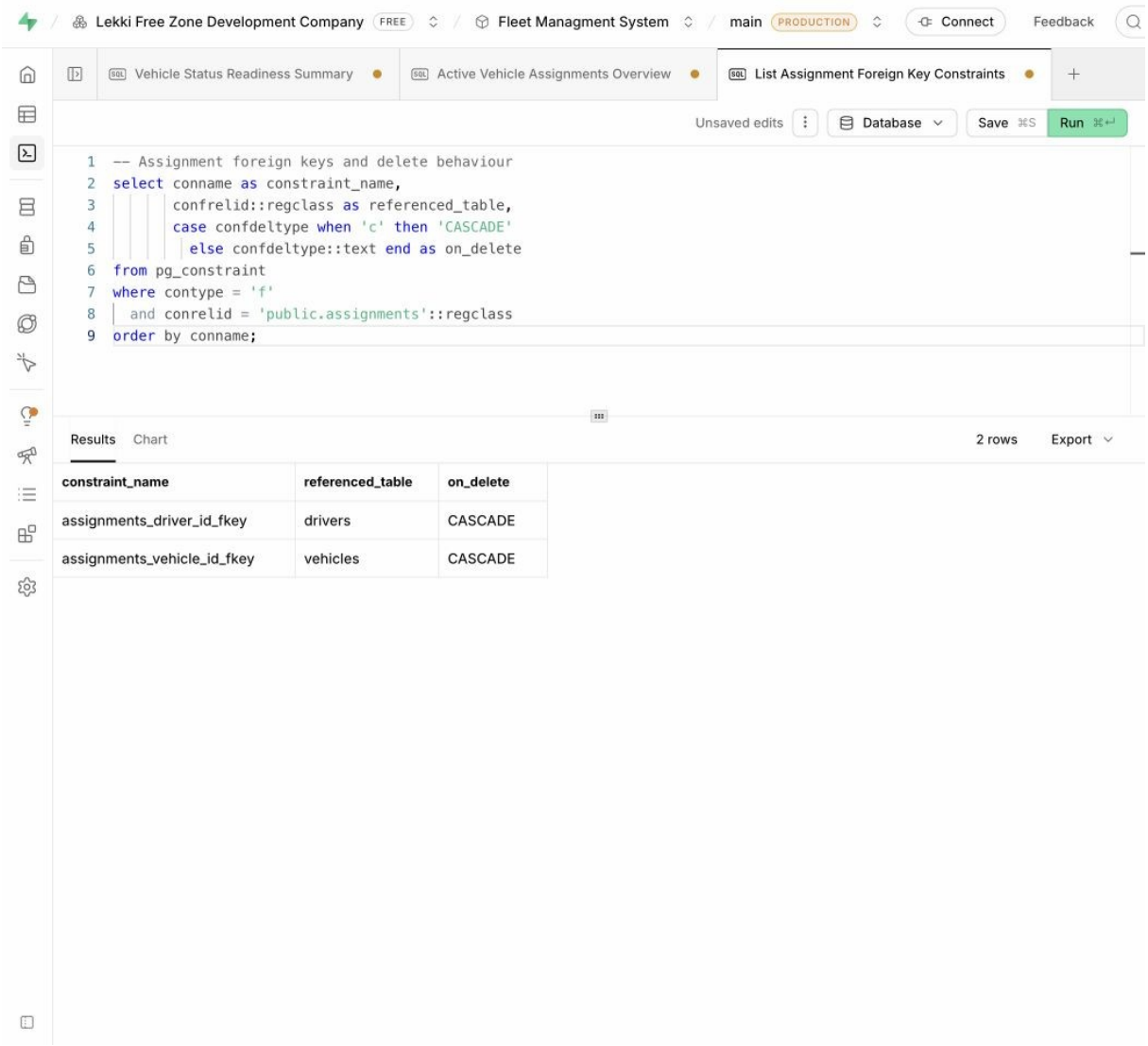


Figure 5.11: Hosted assignment foreign key inspection

Two foreign keys are present: assignments_driver_id_fkey references drivers and assignments_vehicle_id_fkey references vehicles. Both report CASCADE. Deleting a referenced parent can therefore remove its assignment rows. The screenshot inspects that configuration; it does not execute a deletion or prove every integrity case.

5.6.5 Inspect Row Level Security Flags

The pg_tables view exposes whether Row Level Security is enabled. This query checks seven core fleet tables and presents their flags without reading user accounts, credentials or private profile information.

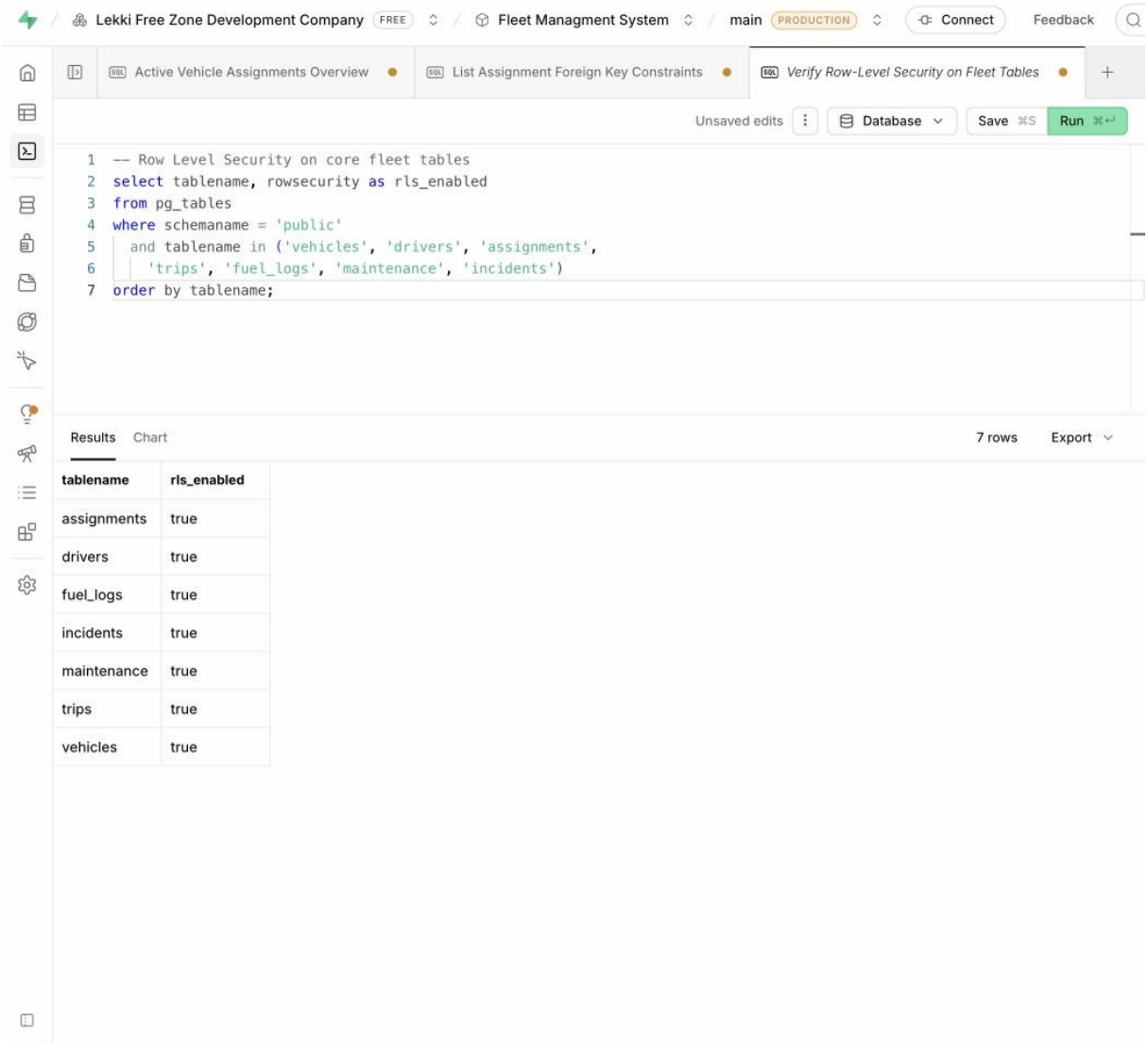


Figure 5.12: Hosted core-table Row Level Security flags

All seven inspected tables return true: assignments, drivers, fuel_logs, incidents, maintenance, trips and vehicles. This verifies that RLS is enabled on this subset of tables. It does not prove that each policy grants or denies the correct application role; authenticated role-by-role tests remain necessary.

CHAPTER SIX

TESTING AND EVALUATION

This chapter records the database verification strategy, representative results, restoration evaluation and limitations.

6.1 Verification Strategy

Verification distinguishes executed local SQL tests from controls that still require hosted integration testing. The Chapter Five walkthrough ran against seven core tables copied from the submission schema, with fictional fixtures and the vehicle timestamp trigger. Automated assertions checked CRUD, search, joins, aggregate totals, duplicate rejection, foreign keys, timestamp updates and delete behaviour. No live project records were changed.

The local engine does not reproduce Supabase Auth or Storage. Its owner-level SQL queries cannot establish whether an authenticated application role passes or fails an RLS policy. Those checks, and a full ten-script restoration, remain separate assessment steps. The test matrix therefore does not mark unexecuted security cases as passed.

6.2 Integrity and Security Tests

ID	Test	Expected result	Outcome
DB01	Insert duplicate vehicle plate	Unique constraint rejects the row.	Local pass
DB02	Insert trip with unknown vehicle	Foreign key rejects the row.	Local pass
DB03	Delete driver referenced by incident	Incident remains and driver_id becomes null.	Local pass
DB04	Update timestamped vehicle	updated_at advances after the update.	Local pass
DB05	End user inserts vehicle	RLS rejects the operation.	Not run
DB06	Editor inserts vehicle	RLS permits the operation.	Not run
DB07	End user changes own role	Authorization trigger raises an exception.	Not run
DB08	Editor uploads document object	Storage policy permits the insert.	Not run
DB09	Editor deletes document object	Storage policy rejects deletion.	Not run
DB10	Repeat notification delivery pair	Composite primary key rejects duplicate.	Not run
DB11	Restore all ten scripts	Schema, policies and fixtures restore successfully.	Not run
DB12	Final submission secret audit	No credentials or private tokens in the delivery.	Pending
DB13	Create, read, search, update, delete	Stored values and row counts match each step.	Local pass
DB14	Two fuel and two service entries	Fuel 50,000; maintenance 25,000; no duplication.	Local pass

6.3 Restoration Evaluation

The package is structured for a new Supabase project rather than a plain PostgreSQL server because profiles reference auth.users and the storage script references storage.objects. Scripts 01 through 09 establish the schema and security model; script 10 adds assessment data. Hosted Auth users must be created separately because password material is intentionally excluded.

The review corrected two order and security issues before this report was produced: document-type write policy creation now occurs after the role helpers exist, and Storage finishes with role-based write policies rather than a broader authenticated-user policy. Import batch creation is also restricted to record managers, and a trigger protects profile authorization fields.

6.4 Limitations

- The public documents bucket is convenient but unsuitable for confidential files without conversion to private signed URLs.
- Status values are mainly validated by the application rather than check constraints or lookup tables.
- Foreign-key indexes and reporting indexes should be selected using production query plans.
- Multi-step assignment and state synchronization are not yet encapsulated in database transactions.
- The package recreates schema and fictional rows but not hosted Auth accounts or binary Storage objects.

CHAPTER SEVEN

ADMINISTRATION AND RESTORATION GUIDE

This chapter explains safe handling, restoration, verification and routine maintenance for assessors and administrators.

7.1 Requirements and Safe Handling

- A Supabase project with access to its SQL Editor.
- The database-submission folder containing scripts 01 through 10.
- A test email address for any Auth account created after restoration.
- No .env.local file, service-role key, SMTP password or personal credential on the flash drive.

7.2 Restoration Procedure

1. Create or select a new Supabase project and wait for the database to become available.
2. Open the SQL Editor and run 01-schema.sql.
3. Run scripts 02 through 07 in numeric order.
4. Run 08-role-access-policies.sql to establish canonical roles and final table write access.
5. Run 09-storage-documents.sql to create the bucket and final object policies.
6. Run 10-sample-data.sql to load fictional demonstration records.
7. Create at least one Auth user in Supabase Authentication.
8. Insert a matching public.profiles row using the same Auth UUID and assign system_admin for assessment administration.
9. Configure the frontend with the project URL and anonymous key, then sign in and verify the registers.

7.3 Verification and Maintenance

Check	Expected result
Tables	Fourteen public application tables are visible.
RLS	Enabled on every application table.
Roles	system_admin, editor and end_user are available.
Storage	documents bucket exists with authenticated read and role-based writes.
Sample data	Vehicle, driver and operational records load without real credentials.
Application	Dashboard and registers return data for a signed-in profile.
Security	An end_user cannot create records or change authorization fields.

Routine maintenance should include backup verification, review of slow queries, index evaluation, monitoring of connection usage, inspection of RLS changes and controlled testing of every migration in a non-production project before deployment.

CHAPTER EIGHT

CONCLUSION AND RECOMMENDATIONS

This chapter summarizes the database outcome and identifies the next improvements.

8.1 Conclusion

The database meets the project objective of centralizing LFZDC fleet records in a relational structure that preserves operational relationships and history. PostgreSQL constraints protect row identity and references, triggers maintain timestamps and authorization fields, and RLS separates read-only users from record managers and system administrators. Supabase provides the surrounding Auth, Storage and API services without changing the underlying relational model.

The ten-script submission package makes the implemented database inspectable and portable for academic assessment. Its fictional data supports meaningful reports and demonstrations without exposing organizational records or private credentials. The audit completed during documentation also improved restoration order and closed policy gaps that could otherwise weaken role separation.

8.2 Recommendations

- Convert the documents bucket to private access and issue short-lived signed URLs for sensitive files.
- Add check constraints or lookup tables for vehicle, driver, maintenance and incident statuses.
- Add measured indexes for foreign keys, dates, unread notifications and document expiry queries.
- Move assignment and synchronized state changes into transactional PostgreSQL functions.
- Add automated database tests for migrations, RLS policies and restoration scripts.
- Establish scheduled backups and perform periodic restoration drills in a separate project.

REFERENCES

[1] PostgreSQL Global Development Group. (2026). PostgreSQL 17 Documentation: Data Definition. https://www.postgresql.org/docs/17/ddl.html

[2] PostgreSQL Global Development Group. (2026). Constraints. https://www.postgresql.org/docs/17/ddl-constraints.html

[3] PostgreSQL Global Development Group. (2026). Row Security Policies. https://www.postgresql.org/docs/17/ddl-rowsecurity.html

[4] PostgreSQL Global Development Group. (2026). Indexes. https://www.postgresql.org/docs/17/indexes.html

[5] Supabase. (2026). Database Overview. https://supabase.com/docs/guides/database/overview

[6] Supabase. (2026). Row Level Security. https://supabase.com/docs/guides/database/postgres/row-level-security

[7] Supabase. (2026). Securing Your Data. https://supabase.com/docs/guides/database/secure-data

[8] Supabase. (2026). Storage Access Control. https://supabase.com/docs/guides/storage/security/access-control

[9] Supabase. (2026). Auth Architecture. https://supabase.com/docs/guides/auth/architecture

[10] Elmasri, R. and Navathe, S. B. (2016). Fundamentals of Database Systems. 7th ed. Pearson.

[11] Silberschatz, A., Korth, H. F. and Sudarshan, S. (2019). Database System Concepts. 7th ed. McGraw-Hill.

[12] LFZDC Fleet Management System database-submission package and backend documentation (project submission, 2026).

APPENDIX A: KEY DATA DICTIONARY

The SQL files remain the complete machine-readable definition of every column. This appendix summarizes the fields most important to identity, relationships, authorization, reporting and audit behaviour.

Table	Column	Type	Constraint	Meaning
profiles	id	uuid	PK/FK auth.users	Application user identity.
profiles	email	text	NOT NULL, UNIQUE	Account email.
profiles	role	app_role	NOT NULL	Authorization role.
profiles	settings	jsonb	DEFAULT '{}'	User preferences.
vehicles	id	text	PK	Vehicle identifier.
vehicles	plate_number	text	NOT NULL, UNIQUE	Registration plate.
vehicles	status	text	NOT NULL	Operational lifecycle state.
vehicles	current_odometer	integer	Nullable	Latest accepted mileage.
vehicles	next_service_due_odometer	integer	Nullable	Mileage-based service threshold.
drivers	id	text	PK	Driver identifier.
drivers	license_number	text	NOT NULL	Driving licence reference.
drivers	license_expiry	date	NOT NULL	Licence expiry date.
assignments	vehicle_id	text	FK vehicles	Assigned vehicle.
assignments	driver_id	text	FK drivers	Assigned driver.
assignments	end_date	date	Nullable	Null indicates active assignment.
fuel_logs	liters	numeric(10,2)	NOT NULL	Fuel volume.
fuel_logs	cost	numeric(12,2)	NOT NULL	Fuel transaction cost.
fuel_logs	odometer	integer	NOT NULL	Mileage at fueling.
maintenance	status	text	NOT NULL	Service workflow state.
maintenance	scheduled_date	date	NOT NULL	Planned service date.
maintenance	cost	numeric(12,2)	NOT NULL	Estimated or recorded cost.
incidents	driver_id	text	FK drivers, nullable	Associated driver when known.
incidents	severity	text	NOT NULL	Incident severity.
incidents	attachments	jsonb	Nullable	Attachment metadata.
trips	distance	numeric(10,2)	NOT NULL	Journey distance.
documents	vehicle_id	text	FK vehicles, nullable	Related vehicle.
documents	driver_id	text	FK drivers, nullable	Related driver.

Table	Column	Type	Constraint	Meaning
documents	file_path	text	Nullable	Storage object path.
documents	expiry_date	date	Nullable	Document expiration.
document_types	name	text	NOT NULL, UNIQUE	Category name.
document_types	applies_to	text	CHECK	vehicle, driver or both.
notifications	is_read	boolean	DEFAULT false	Read status.
notifications	vehicle_id	text	FK vehicles, nullable	Related vehicle.
notifications	driver_id	text	FK drivers, nullable	Related driver.
notification_email_deliveries	notification_id	text	Composite PK/FK	Delivered notification.
notification_email_deliveries	profile_id	uuid	Composite PK/FK	Recipient profile.
vehicle_odometer_logs	reading	integer	NOT NULL	Recorded mileage.
vehicle_odometer_logs	source	text	NOT NULL	manual, trip, fuel or maintenance.
import_batches	created_counts	jsonb	DEFAULT '{}'	Created rows by entity.
import_batches	issues	jsonb	DEFAULT '[]'	Row-level validation issues.
import_batches	sheet_summaries	jsonb	DEFAULT '[]'	Per-sheet import outcome.

APPENDIX B: RESTORATION SCRIPT MANIFEST

File	Creates or changes	Depends on
01-schema.sql	Core schema and read access	Supabase auth schema
02-vehicle-odometer.sql	Mileage field and log table	vehicles
03-vehicle-service-mileage.sql	Service mileage field	vehicles
04-document-types.sql	Document category table	pgcrypto extension
05-document-file-path.sql	Document object path	documents
06-profile-settings.sql	Profile JSONB settings	profiles
07-notification-email-deliveries.sql	Delivery audit table	notifications, profiles
08-role-access-policies.sql	Roles, safeguards and final writes	all application tables
09-storage-documents.sql	Bucket and object policies	Storage schema, role helpers
10-sample-data.sql	Fictional operational rows	scripts 01 through 09

The scripts are idempotent where practical through IF NOT EXISTS, DROP POLICY IF EXISTS and ON CONFLICT clauses. They should still be tested in a fresh project before assessment because enum additions and data migrations are state-dependent operations.

APPENDIX C: SAMPLE SQL QUERIES

C.1 List Active Vehicles and Current Drivers

```
select v.plate_number, v.brand, v.model, d.full_name
from public.vehicles v
left join public.drivers d on d.id = v.current_driver_id
where v.status = 'Active'
order by v.plate_number;
```

C.2 Monthly Fuel Cost by Vehicle

```
select v.plate_number, sum(f.cost) as monthly_fuel_cost
from public.fuel_logs f
join public.vehicles v on v.id = f.vehicle_id
where f.date >= date_trunc('month', current_date)::date
group by v.id, v.plate_number
order by monthly_fuel_cost desc;
```

C.3 Outstanding Maintenance

```
select v.plate_number, m.service_type, m.status, m.scheduled_date, m.cost
from public.maintenance m
join public.vehicles v on v.id = m.vehicle_id
where m.status in ('Scheduled', 'In Progress', 'Overdue')
order by m.scheduled_date;
```

C.4 Expiring Driver Licences

```
select full_name, license_number, license_expiry
from public.drivers
where license_expiry between current_date and current_date + interval '90 days'
order by license_expiry;
```

C.5 Suggested Indexes for Future Tuning

These optional statements are recommendations, not changes applied by the submission scripts. Test query plans before introducing them.

```
create index if not exists idx_fuel_logs_vehicle_date
  on public.fuel_logs (vehicle_id, date desc);

create index if not exists idx_documents_expiry
  on public.documents (expiry_date)
  where expiry_date is not null;
```